Reproduction report

"Optimization Driven Quantum Circuit Reduction"
(Rosenhahn, Osborne, Hirche, arXiv:2502.14715)

Reproduction based on the authors' MATLAB demo code, ported to Python

Summary

Claim	Status	Comment
Table I (compute-graph sizes)	**reproduced**	All 22 node/edge counts identical.
Fig. 1 (Bell example)	**reproduced**	Both circuits equal CNOT$(H \otimes I)$; 4 gates is optimal.
Fig. 4 (40-gate 2q example)	**reproduced**	$40 \to 4$. But the paper's own 4-gate output has optimum 2.
Fig. 6/7, Table II (V1 > V2 > V3 time)	**partly reproduced**	Ordering V1 $\gg$ V2 holds. V3 beats V2 *only* with a tuned forest threshold; at the default 0.5 it is slower than V2.
Table III (ion trap 3q)	**reproduced**	With the 24-op depth-5 database: 47.1 gates vs. paper 45.
Table IV/V (IBM-style 3q/15q)	**partly reproduced**	3q: reproduced and stronger (36.8 vs. paper 72). 15q: the method only ties Qiskit 2.5 L3 (335.9 vs. 332.1).
NJP 4q statistics	**reproduced**	Faithful port: 78.8 / 76.0 gates (published 111 / 107); Qiskit L3: 164.9 / 140.9.
Fig. 9 (IBM hardware)	**partly reproduced**	Noisy-simulator reproduction: the effect exists at transpiler levels 0/1 and essentially vanishes at level 3.

1 What "the code given" is, and how we ran it

The authors ship a MATLAB demo, linked only in the journal version's ref. [38] (`https://www.tnt.uni-hannover.de/en/staff/rosenhahn/QCOptimDemo.zip`; the arXiv version has no link). It is `QCOptimDemo`, with `FullQCGraphIOn3_4.m`, `optimCodeGMode1DComp.m` and helpers). It implements the compute-graph database and the V2 optimiser only; V1 and V3 are described in the paper but not included. The demo requires MATLAB's Quantum Computing Support Package, which is not available to us (and is not Octave compatible). We therefore ported it line by line to Python (`qcr`):

- `gates.py`: MATLAB gate conventions ($R_P(\theta) = e^{-i\theta P/2}$, qubit 1 most significant, $U = O(L)\cdots O(1)$), pools in the order of `QoperatorsIon1.m`.
- `unitary.py`: `compare.m` verbatim (least-squares phase, spectral norm), plus a canonical phase-normalised hash key.
- `compute_graph.py`: the full compute graph (BFS), parent pointers = shortest-path factorisations.
- `reducer.py`: the optimiser loop of `optimCodeGMode1DComp.m`: shuffle w.p. 0.7 (each commuting pair w.p. 0.6), block start/length sampling identical to MATLAB's `randi` calls, $\leq$ 3-wire rejection, wire reduction/lift, `simplereduce` including its loop bounds, tolerance 10^{-3}, database without the identity. V2 lookups exist in two modes: `scan` (the MATLAB linear scan over

entries shorter than the block) and `hash` (an $O(1)$ dictionary; same replacements, used where only output quality matters).
- V1 (random sampling, 300 candidate words per block) and V3 (random forest, 50 trees, features = remapped token sequence + token histogram + length; labels from the exact database, as the paper describes) were implemented from the paper's text.

Every reduced circuit in every experiment was checked for equivalence with its input (dense unitary for $n \leq 5$, random-state test for $n = 15$). **All checks passed**; the maximal unitary deviation was $\sim 10^{-12}$. The authors' own demo output (`longcode10`→`shortcode10`, 300 → 140 gates, 5 qubits) is also exactly equivalent once the 4-decimal angles in the QASM file are snapped to $\pm\pi/2$.

Hardware: 8-core laptop, Python 3.14, NumPy 2.3, Qiskit 2.5, qiskit-aer 0.17, qiskit-ibm-runtime fake backends. Timing experiments ran 8 processes in parallel; all four variants of a given circuit ran concurrently, so relative comparisons are fair, but absolute times are not comparable with the authors' MATLAB numbers.

2 Table I: compute graphs

Node counts for 2 qubits / 14 operators (depth 1–6) and 3 qubits / 24 operators (depth 1–5) are *identical* to the paper, and so are the edge counts (which follow $E(d) = N(d-1) \cdot |\mathcal{OP}|$). The operator counts 14/24 imply the pool contains RXX($\pm\pi/2$). The demo code uses only $+\pi/2$ (13/21 operators), which gives smaller graphs (3 qubits, depth 4: 12,441 nodes). Building the largest graph (137,572 nodes) takes 8 s in Python. An independent exact engine (Clifford tableaux, see the extension paper) gives the same counts.

3 Figures 1, 4, 5: worked examples

- Fig. 1: both the 8-gate naive and the 4-gate optimised Bell circuits equal CNOT $(H \otimes I)$ to 10^{-13}. Exhaustive search shows no 3-gate circuit exists.
- Fig. 4 analogue (2 qubits, 40 random gates, 2-qubit depth-6 database): $40 \to 34 \to 26 \to 24 \to 20 \to 18 \to 16 \to 8 \to 4$. The paper's figure also ends at 4. However, the paper's displayed final circuit $\mathrm{RXX}(\frac{\pi}{2})\mathrm{RXX}(\frac{\pi}{2})\mathrm{RZ}_2(\frac{\pi}{2})\mathrm{RZ}_2(-\frac{\pi}{2})$ contains an inverse pair that `simplereduce` cannot see (it never checks the last pair), and the database cannot remove it (identity excluded). Its optimum is 2. Note also that every step in our trace removes an even number of gates. For two qubits this is forced (gate-count parity is invariant, see the extension paper). The paper's Fig. 4 shows a $6 \to 3$ replacement, so it must have used a pool different from the one stated.
- Fig. 5 analogue (3 qubits, 200 gates): $200 \to 36$ in 31 replacements.
- Authors' demo circuit (`longcode10`, 5 qubits, 300 gates): authors' output 140 gates. Our port with the demo's 21-op depth-4 database gives 140 (range 132–148); with the 24-op depth-5 database 115 (range 112–117) (5 seeds each).

4 Table II, Figs. 6–7: V1 vs. V2 vs. V3

Protocol (unspecified in the paper; we follow the demo): 100 random 5-qubit ion-trap circuits of length 100, 3-qubit depth-4 database (12,441 entries; linear-scan lookup ≈10 ms), target length 60, cap 600 s. V3 is run with the standard majority-vote threshold (0.5) and with a recall-oriented threshold (0.2).

Variant	runs	mean (s)	std (s)	median (s)	reached	attempts	lookups
V1-RS	100	254.0	242.3	134.2	73	51,350	51,350
V2-DR	100	46.4	129.8	6.5	96	7,904	7,904
V3-RF	100	171.8	241.6	16.9	79	276,445	915
V3-RF(t=0.2)	100	33.7	108.4	5.3	98	10,553	1,216

Table 1: Our Table II. Paper: V1 199±352 s, V2 55±96 s, V3 38±40 s. "reached" = runs that hit the target within 600 s.

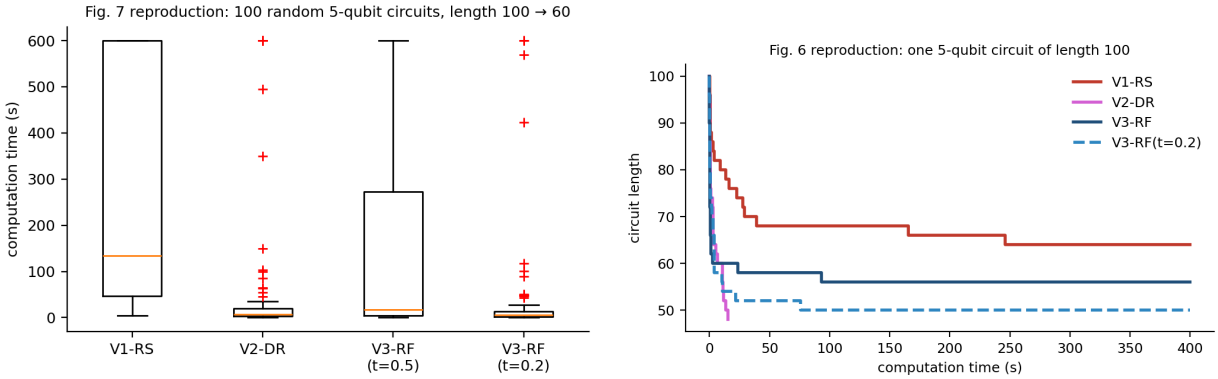


Figure 1: Left: Fig. 7 reproduction. Right: Fig. 6 reproduction (single circuit, target 48, cap 400 s). On this circuit V2 reaches the target in 15 s, while V1 (64), V3 at threshold 0.5 (56) and V3 at 0.2 (50) stall. Single-circuit curves are anecdotal; the 100-circuit statistics on the left are the meaningful comparison.

Findings. (i) V1 is clearly slowest and has the heaviest tail, as in the paper. (ii) The database (V2) gives a large speed-up. (iii) The forest's recall decides V3. On held-out blocks it predicts "reducible" for only 30% of truly reducible blocks at threshold 0.5 (92% at 0.2). At 0.5 it throws away most reductions and needs ∼40× more attempts, and so is *slower* than V2. At 0.2 it is the fastest variant, as the paper reports. The paper does not report the threshold or the classifier's accuracy, so its V3 result can be reproduced only after tuning. (iv) Replacing the linear scan by a hash lookup (0.03 ms instead of 10 ms) makes V2 faster than any V3 configuration, removing the reason for V3.

5 Tables III–V: comparison with Qiskit

Circuits are uniformly random native-gate strings (this matches the paper's gate proportions). "Ours" = faithful V2 loop with hash lookup, stopped after 100,000 consecutive failed attempts (or 400 s). Database: ion trap, 24-op depth 5 (Table I); IBM-style (RX, RZ($\pm\frac{\pi}{2}$), CZ), 15-op depth 6 (109,639 entries). With the 21-op depth-4 demo database, the 3-qubit ion-trap result stalls at ≈ 76 gates instead of ≈ 42. The paper does not state which database was used, but only the larger one reproduces its numbers. Columns: first circuit (as in the paper), then mean±std over all circuits.

Method	circuit 1 rx	ry	rz	rxx	total	mean ± std over 20 circuits rx	ry	rz	rxx	total
original	89	86	83	42	300	86±9	84±9	87±7	43±7	300.0±0.0
qiskit L1	27	35	37	42	141	20±5	36±5	36±5	43±7	135.3±16.0
qiskit L2	24	34	36	29	123	32±6	31±6	38±6	31±4	131.3±14.2
qiskit L3	19	35	37	28	119	30±7	30±6	37±6	30±4	127.2±16.5
ours	3	11	13	17	44	3±1	12±4	13±4	19±5	47.1±11.7

Table 2: Table III reproduction (ion trap, 3 qubits, 300 gates). Paper: original 300, Qiskit L3 176, ours 45.

Method	circuit 1 rx	rz	cz	total	mean ± std rx	rz	cz	total
Table IV: 3 qubits, 300 gates (paper: Qiskit L3 153, ours 72)								
original	127	115	58	300	119±9	121±9	60±7	300.0±0.0
qiskit L1	58	53	56	167	52±6	54±6	49±6	154.7±15.1
qiskit L2	55	34	35	124	47±6	29±4	31±5	106.0±13.2
qiskit L3	54	33	34	121	46±6	28±4	30±5	104.0±13.0
ours	16	8	13	37	17±7	6±3	14±5	36.8±14.6
Table V: 15 qubits, 500 gates (paper: Qiskit L3 313, ours 293)								
original	86	84	330	500	90±7	88±7	322±11	500.0±0.0
qiskit L1	75	74	314	463	74±5	75±4	305±8	453.9±7.3
qiskit L2	78	35	233	346	76±6	43±6	214±15	332.1±19.9
qiskit L3	78	35	233	346	76±6	43±6	214±15	332.1±19.9
ours	79	37	236	352	78±5	45±6	213±16	335.9±20.5

Table 3: Tables IV and V reproduction (IBM-style gate set).

Findings. (i) *Table III reproduces*: the faithful loop reaches 47.1 gates on average (paper: 45 on its single circuit), against 127 for Qiskit L3. (ii) *Table IV*: we obtain 36.8 gates (paper: 72), so the method is even stronger than reported here. (iii) Qiskit 2.5 is much stronger than the version used in the paper (L3: 127 vs. 176 and 104 vs. 153). (iv) *Table V does not reproduce*: at 15 qubits the method (335.9 gates, 213 CZ) only ties Qiskit L3 (332.1 gates, 213.5 CZ), whereas the paper reports 293 vs. 313. Most random 15-qubit gate pairs act on disjoint wires, so there is little for 3-wire windows to do that Qiskit's commutation analysis does not already do. These runs also reached the 400 s cap, so a longer budget could help somewhat. (v) Two-qubit gates are reduced proportionally (3q ion: 43 → 18.9 RXX; Qiskit L3: 29.9). All outputs were verified equivalent to their inputs.

Method	circuit 1 rx	ry	rz	rxx	total	mean ± std over 30 circuits rx	ry	rz	rxx	total
original	86	79	77	58	300	78±8	79±8	81±7	62±7	300.0±0.0
qiskit L1	29	35	38	58	160	28±6	39±5	44±5	62±7	172.0±14.4
qiskit L2	45	26	48	46	165	37±6	36±5	45±6	48±7	166.0±14.4
qiskit L3	45	26	48	46	165	37±6	36±5	44±6	47±7	164.9±14.2
ours	2	15	19	29	65	5±2	21±4	20±6	33±7	78.8±15.7

Method	rx	rz	cz	total	rx	rz	cz	total
original	109	106	85	300	106±8	111±8	83±8	300.0±0.0
qiskit L1	58	53	67	178	58±5	63±7	70±8	191.0±15.7
qiskit L2	64	41	49	154	58±6	36±4	48±6	141.7±14.7
qiskit L3	64	41	49	154	58±6	36±4	47±6	140.9±15.4
ours	23	8	27	58	33±8	10±2	32±8	76.0±16.8

Table 4: Published (NJP Tables 6/7) "ours": ion trap 10/29/29/43 (total 111); IBM-style 45/19/43 (total 107).

NJP protocol (4 qubits, length 300; 30 random circuits per gate set, the first 30 of the 100 used in the extension paper). Findings. Our faithful port gives 78.8 (ion trap) and 76.0 (IBM-style) gates on average, *better* than the published 111 and 107. Qiskit 2.5 L3 gives 164.9

and 140.9. The difference to the published numbers is plausibly due to the database used (ours: 24-op depth 5 / 15-op depth 6, unstated in the paper) and run time. The qualitative claim, that the method clearly beats Qiskit on few-qubit random circuits, reproduces. The extension paper shows that an exact convex-window optimiser reaches 56.7 / 59.1 on the same benchmark in ∼6 s per circuit.

6 Fig. 9: hardware experiment (noisy simulation of the same chips)

We cannot access IBM Brisbane/Kyiv (both retired), so we use Qiskit's calibrated noise models `FakeBrisbane` and `FakeKyiv`. Unlike the paper we use 30 random 2-qubit circuits of length 40 (each reduced with the 2-qubit database), 4000 shots, and three transpiler levels. Metric: total-variation distance (TVD) of the measured distribution from the ideal one.

Chip	level	gates long/short	transpiled ops	2q gates (ECR)	TVD long	TVD short	short better
Brisbane	0	40 / 5.4	311 / 97	6.7 / 3.1	0.059±0.008	0.041±0.005	22/30
Brisbane	1	40 / 5.4	37 / 18	6.7 / 3.1	0.046±0.006	0.037±0.005	17/30
Brisbane	3	40 / 5.4	14 / 15	1.1 / 1.2	0.018±0.002	0.019±0.002	8/30
Kyiv	0	40 / 5.4	231 / 59	6.7 / 3.1	0.024±0.003	0.018±0.001	20/30
Kyiv	1	40 / 5.4	41 / 22	6.7 / 3.1	0.021±0.002	0.018±0.001	17/30
Kyiv	3	40 / 5.4	14 / 15	1.1 / 1.2	0.012±0.001	0.012±0.001	11/30

Table 5: Fig. 9 reproduction, mean over 30 circuits (± standard error).

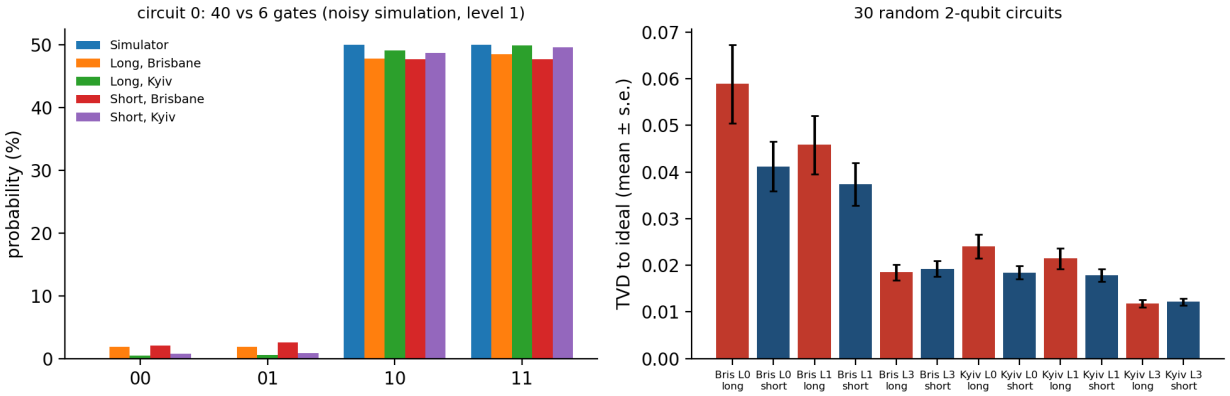


Figure 2: Left: Fig. 9-style bars for one circuit. Right: TVD statistics.

Findings. At transpiler levels 0 and 1 the reduced circuits are closer to the ideal distribution on both chips (Brisbane L0: TVD 0.059 → 0.041, better on 22/30 circuits; Kyiv L0: 0.024 → 0.018, 20/30), and the newer Kyiv calibration is less noisy than Brisbane. Both observations match the paper's Fig. 9. At level 3, however, Qiskit consolidates every 2-qubit block and re-synthesises it, so long and short circuits compile to the same ≈ 1.1 two-qubit gates and the advantage vanishes (Brisbane 0.018 vs. 0.019; Kyiv 0.012 vs. 0.012). Plain Qiskit level 3 on the *long* circuit beats the reduced circuit at level 1 (0.018 vs. 0.037 on Brisbane). The paper's hardware evidence therefore depends on an unreported transpiler setting. Its native-op counts (156 vs. 34 on Brisbane) are consistent with level 0/1 compilation by an older Qiskit. Our reduced circuits average 5.4 gates (paper: 8).

7 Bugs and undocumented choices found while reproducing

1. `simplereduce.m` never tests the last adjacent pair, and skips the pair following a deletion (visible in the paper's Fig. 4).

2. The database omits the identity (`Dists>0`), so identity blocks cannot be deleted by lookups.
3. Tolerances differ: paper 10^{-5}, optimiser 10^{-3}, graph builder 10^{-4}.
4. Operator pools differ between Table I (14/24 ops) and the demo (13/21 ops), and the database used for Tables III–V is unstated. Only the larger one reproduces Table III.
5. Unspecified in the paper: qubit count and target length for Table II; forest features, size, threshold and accuracy; V1 sampling budget; transpiler level and shot count for the hardware experiment; which Qiskit version was used.
6. The hardware section's text names "RX, RY and CX", but the tables report RX, RZ, CZ.
7. The graph builder recomputes BFS distances only with probability 0.3 per step. This does not change the final counts.

8 How to rerun

```
cd experiments
python3 exp_figures.py table1 examples demo convergence
python3 exp_table2_stats.py          # Table II / Fig. 7 (~2-3 h on 8 cores)
python3 exp_quality.py               # Tables III-V + NJP protocol
python3 exp_hardware.py              # Fig. 9 with FakeBrisbane / FakeKyiv
python3 make_assets.py               # tables/figures used in this report
```